MODUL RINGKASAN & CHEATSHEET

Sistem Operasi

Konsep inti, rumus, contoh hitung, dan referensi praktikum dalam satu modul.

Berdasarkan *Operating System Concepts* 10th ed. (Silberschatz, Galvin, Gagne) dan materi praktikum C & Bash

BAGIAN I · KONSEP

01 Pengantar Sistem Operasi
02 Struktur Sistem Operasi
03 Proses
04 Thread dan Concurrency
05 CPU Scheduling
06 Synchronization Tools
07 Contoh Sinkronisasi
08 Deadlock
18 Virtual Machine

BAGIAN II · PRAKTIKUM

P1 Bahasa C — Dasar
P2 Bahasa C — Pointer, Array, fork
P3 Bash Scripting
P4 grep, sed, awk

Disusun oleh **Michael Evan**

Oktober 2026

Daftar Isi

BAGIAN I · KONSEP SISTEM OPERASI

01 Pengantar Sistem Operasi 4
Interrupt · I/O dan DMA · Storage hierarchy · Operasi OS · Tanggung jawab OS per area · Arsitektur sistem · Lingkungan komputasi · Struktur data kernel · Tambahan untuk pertanyaan pemantik

02 Struktur Sistem Operasi 7
Layanan OS · System call · Enam jenis system call · System program dan background service · Linker dan loader · Desain dan implementasi · Struktur kernel · Build dan boot · Debugging dan performa

03 Proses 11
Isi proses di memori · State proses · Process Control Block (PCB) · Scheduling dan context switch · Membuat dan mengakhiri proses · IPC (Interprocess Communication) · Detail message passing · Contoh sistem IPC · Komunikasi client–server

04 Thread dan Concurrency 15
Manfaat multithreading · Concurrency vs parallelism · Amdahl's Law · User thread, kernel thread, dan model pemetaan · Pthreads (POSIX, IEEE 1003.1c) · Implicit threading · Threading issues · Contoh OS

05 CPU Scheduling 18
Konsep dasar · Kriteria dan rumus · Ringkasan algoritma · Contoh hitung dengan Gantt chart · Prediksi burst untuk SJF · Round Robin: memilih q · Multilevel feedback queue · Thread scheduling · Multiprocessor scheduling · Real-time scheduling · Contoh OS · Evaluasi algoritma

06 Synchronization Tools 23
Critical-section problem · Solusi awal dan kenapa kurang · Peterson's Solution (2 proses) · Memory model dan memory barrier · Instruksi atomik hardware · Mutex lock · Semaphore · Monitor dan condition variable · Ringkasan alat · Liveness

07 Contoh Sinkronisasi 27
Bounded buffer (n slot) · Readers-writers · Dining philosophers (5 filsuf, 5 sumpit) · Sinkronisasi di kernel · API POSIX (C, -pthread) · Java · Pendekatan alternatif

08 Deadlock 31
Model sistem · Empat kondisi perlu (Coffman) · Resource-allocation graph (RAG) · Deadlock prevention · Deadlock avoidance · Banker's algorithm · Contoh Banker (A = 10, B = 5, C = 7) · Deadlock detection · Recovery

18 Virtual Machine 35
Definisi formal (IBM, 1972) · Jenis VMM · Manfaat · Building blocks · Siklus hidup VM · Virtualisasi dan komponen OS · Live migration · Contoh · Riset virtualisasi

BAGIAN II · PRAKTIKUM

P1 Bahasa C — Dasar 39
Setup dan alur kompilasi · Struktur program · Tipe data dan format specifier · Input dengan scanf · Percabangan · Perulangan · Kunci latihan mandiri

P2 Bahasa C — Pointer, Array, fork 42
Pointer · Array dan aritmetika pointer · fork(), exec(), wait() · Menghitung jumlah proses

P3 Bash Scripting 44
Cek shell dan perintah dasar · Struktur dan cara menjalankan · Variabel dan input · Percabangan · Perulangan · Fungsi dan argumen · Contoh praktis: backup otomatis · Tips

P4 grep, sed, awk 47
grep · sed · awk · Menggabungkan dengan pipe

Cara membaca modul ini

- **Intisari** di awal tiap bab merangkum inti bab dalam 2–3 kalimat.
- **Catatan** berwarna oranye menandai jebakan atau kesalahan umum.
- Blok kode bisa langsung dicoba; komentar dalam bahasa Indonesia.

Contoh hitung (Gantt chart, Banker, deteksi deadlock) sudah dicek ulang angkanya.

BAGIAN I

Konsep Sistem Operasi

Ringkasan bab 1–8 dan 18 dari slide kuliah: struktur OS, proses dan thread, penjadwalan CPU, sinkronisasi, deadlock, dan virtualisasi.

01 Pengantar Sistem Operasi

02 Struktur Sistem Operasi

03 Proses

04 Thread dan Concurrency

05 CPU Scheduling

06 Synchronization Tools

07 Contoh Sinkronisasi

08 Deadlock

18 Virtual Machine

BAB 1

01 Pengantar Sistem Operasi

INTISARI

OS adalah program perantara antara user dan hardware: ia mengalokasikan resource (*resource allocator*) dan mengendalikan eksekusi program (*control program*). Bagian yang selalu berjalan disebut **kernel**; sisanya adalah system program, application program, dan middleware.

Empat komponen sistem komputer, dari bawah ke atas: hardware (CPU, memori, I/O) → OS → application program → user.

Interrupt

- Tiap device controller punya local buffer dan dikelola OS lewat device driver. Selesai I/O, controller memberi tahu CPU dengan **interrupt**.
- Interrupt memindahkan kontrol ke *interrupt service routine* (ISR) lewat **interrupt vector**, yaitu tabel alamat semua ISR. Alamat instruksi yang terputus wajib disimpan.
- **Trap/exception** = interrupt dari software, karena error (mis. bagi nol) atau permintaan layanan OS (system call).
- OS bersifat *interrupt driven*: ia baru bekerja kalau ada interrupt atau trap.

Alur penanganan interrupt:

1. Device selesai, controller mengirim sinyal interrupt.
2. CPU menyimpan register dan program counter.
3. CPU menentukan jenis interrupt dan lompat ke ISR lewat interrupt vector.
4. ISR selesai, state dipulihkan, program yang terputus lanjut.

I/O dan DMA

- **Synchronous I/O**: kontrol kembali ke program setelah I/O selesai. CPU menganggur (wait instruction/loop), hanya satu I/O sekaligus.
- **Asynchronous I/O**: kontrol langsung kembali ke program. OS mencatat status tiap device di *device-status table*.
- **DMA (Direct Memory Access)**: controller memindahkan satu blok data langsung antara buffer dan memori tanpa CPU. Hanya satu interrupt per blok, bukan per byte. Dipakai untuk device cepat.

Storage hierarchy

Makin ke bawah makin lambat, makin besar, makin murah per bit. Main memory adalah satu-satunya storage besar yang bisa diakses CPU secara langsung.

Level	Teknologi	Volatile?
Registers	di dalam CPU	ya
Cache	SRAM	ya
Main memory	DRAM	ya
Nonvolatile memory	SSD / flash	tidak
Hard disk drive	platter magnetik (track → sector)	tidak
Optical disk	CD / DVD / Blu-ray	tidak
Magnetic tape	pita	tidak

- **Caching** = menyalin data yang sedang dipakai ke storage lebih cepat. Cache dicek dulu; kalau tidak ada, data disalin ke cache. Masalah desain: ukuran cache dan *replacement policy*.
- Di sistem multiprocessor, hardware wajib menjamin **cache coherency** (semua CPU melihat nilai terbaru).
- Satuan: KB = 1024 B, MB = 1024^2 B, GB = 1024^3 B, TB = 1024^4 B. Jaringan diukur dalam bit.

Operasi OS

- **Boot**: bootstrap program menginisialisasi sistem → load kernel → kernel menjalankan system daemons.
- **Multiprogramming (batch)**: beberapa job ada di memori. Saat satu job menunggu I/O, CPU pindah ke job lain, sehingga CPU selalu sibuk.
- **Multitasking (time sharing)**: CPU berganti job sangat sering sehingga user bisa berinteraksi; response time < 1 detik. Butuh CPU scheduling, swapping, dan virtual memory.
- **Dual mode**: hardware menyediakan *mode bit* (kernel = 0, user = 1). System call mengubah ke kernel mode; return mengembalikan ke user mode. *Privileged instruction* hanya bisa jalan di kernel mode.
- **Timer**: counter dikurangi oleh clock; saat nol terjadi interrupt dan OS mengambil alih. Mencegah infinite loop atau proses memonopoli CPU. Mengatur timer adalah privileged instruction.

Tanggung jawab OS per area

Program = entitas pasif; proses = program yang sedang berjalan (aktif).

Area	Aktivitas OS
Proses	buat/hapus, suspend/resume, sinkronisasi, komunikasi, penanganan deadlock
Memori	lacak bagian yang dipakai dan oleh siapa, pilih apa yang masuk/keluar memori, alokasi/dealokasi
File system	buat/hapus file dan direktori, operasi dasar file, mapping ke secondary storage, backup
Mass storage	mount/unmount, free-space management, alokasi, disk scheduling, partisi, proteksi
I/O subsystem	buffering, caching, spooling, antarmuka device driver

- **Protection** = mekanisme mengontrol akses proses/user ke resource. **Security** = pertahanan dari serangan internal/eksternal (DoS, worm, virus, pencurian identitas).
- Akses diatur lewat user ID dan group ID. *Privilege escalation* = pindah ke effective ID dengan hak lebih besar.

Arsitektur sistem

- **Multiprocessor** (parallel / tightly coupled): throughput naik, *economy of scale*, reliability (graceful degradation, fault tolerance).
  - *Asymmetric*: tiap CPU punya tugas khusus. *Symmetric (SMP)*: semua CPU mengerjakan semua tugas.
- **Multicore**: beberapa core dalam satu chip; lebih hemat daya dan komunikasi antar-core lebih cepat.
- **NUMA**: tiap CPU punya memori lokal; akses ke memori CPU lain lebih lambat.
- **Clustered system**: beberapa sistem bekerja sama, biasanya berbagi storage lewat SAN.
  - *Asymmetric cluster*: satu mesin *hot-standby*. *Symmetric cluster*: semua node jalan dan saling memantau.
  - Dipakai untuk high availability atau HPC; DLM mencegah operasi yang bertabrakan.

Lingkungan komputasi

- **Traditional**, **mobile** (iOS, Android; GPS, giroskop), **client-server** (compute server, file server), **peer-to-peer** (semua node setara, mis. Skype).
- **Cloud**: public, private, hybrid. Layanan: SaaS (aplikasi), PaaS (software stack, mis. database), IaaS (server/storage).
- **Real-time embedded**: batas waktu tetap; operasi dianggap benar hanya jika constraint waktu terpenuhi.
- **Distributed system**: sistem terpisah yang terhubung jaringan (LAN, WAN, MAN, PAN); network OS memberi ilusi satu sistem.
- **Emulation** (CPU sumber ≠ target, paling lambat) vs **virtualization** (guest OS dikompilasi native untuk CPU yang sama, dikelola VMM).
- **FOSS**: source code tersedia. FSF memperkenalkan GPL (*copyleft*). Contoh: GNU/Linux, BSD UNIX.

Struktur data kernel

- Linked list (singly, doubly, circular), binary search tree (worst case O(n), balanced O(lg n)), hash map, bitmap (n bit untuk status n item).
- Di Linux: `<linux/list.h>`, `<linux/kfifo.h>`, `<linux/rbtree.h>`.

Tambahan untuk pertanyaan pemantik

> **CATATAN**
> Bagian ini di luar slide, untuk menjawab pertanyaan pembuka dosen.

- **POSIX** = standar API IEEE untuk OS mirip Unix. macOS tersertifikasi, Linux hampir sepenuhnya patuh, Windows tidak native (lewat WSL/Cygwin).
- **SDRAM → DDR → DDR2 → DDR3**: DDR mengirim data di dua sisi clock; tiap generasi menaikkan prefetch (2n → 4n → 8n) dan menurunkan tegangan (2,5 V → 1,8 V → 1,5 V).

02 BAB 2

Struktur Sistem Operasi

INTISARI

OS menyediakan layanan untuk user dan program; program mengaksesnya lewat **system call**, biasanya dibungkus API. Kernel bisa disusun monolithic, layered, microkernel, modular, atau hybrid, dan OS modern hampir selalu hybrid.

Layanan OS

- **Untuk user**: user interface (CLI, GUI, touchscreen, batch), program execution, I/O operations, file-system manipulation, communications (shared memory atau message passing), error detection.
- **Untuk efisiensi sistem**: resource allocation, logging (siapa pakai apa dan berapa), protection and security (kontrol akses + autentikasi).
- **CLI/shell** mengambil perintah lalu menjalankannya. Perintah bisa built-in atau sekadar nama program, sehingga fitur baru tidak perlu mengubah shell.

System call

- System call = antarmuka pemrograman ke layanan OS, biasanya ditulis dalam C/C++.
- Program jarang memanggil system call langsung; ia memakai **API**: Win32 (Windows), POSIX (UNIX, Linux, macOS), Java API (JVM).
- Tiap system call punya **nomor**; system-call interface memakai tabel berindeks nomor itu, memanggil fungsi di kernel, lalu mengembalikan status dan return value. Pemanggil tidak perlu tahu implementasinya.
- Contoh: `printf()` (library call) di dalamnya memanggil `write()` (system call).

Tiga cara mengirim parameter ke OS:

1. **Register** — paling sederhana, tapi jumlah register terbatas.
2. **Block/table di memori**, alamatnya dikirim lewat register (Linux, Solaris).
3. **Stack** — program push, OS pop.

Cara 2 dan 3 tidak membatasi jumlah atau panjang parameter.

Enam jenis system call

Jenis	Contoh operasi	Windows	Unix
Process control	create/terminate, load/ execute, wait, alokasi memori, lock	`CreateProcess()` `ExitProcess()` `WaitForSingleObject()`	`fork()` `exit()` `wait()`
File management	create/delete, open/close, read/write, atribut	`CreateFile()` `ReadFile()` `WriteFile()` `CloseHandle()`	`open()` `read()` `write()` `close()`
Device management	request/release, read/write, attach/detach	`SetConsoleMode()` `ReadConsole()`	`ioctl()` `read()` `write()`
Information maintenance	waktu/tanggal, data sistem, atribut	`GetCurrentProcessID()` `SetTimer()` `Sleep()`	`getpid()` `alarm()` `sleep()`
Communications	buat koneksi, kirim/terima pesan, shared memory	`CreatePipe()` `CreateFileMapping()` `MapViewOfFile()`	`pipe()` `shm_open()` `mmap()`
Protection	atur izin, izinkan/tolak user	`SetFileSecurity()` `InitializeSecurityDescriptor()`	`chmod()` `umask()` `chown()`

Contoh FreeBSD: shell memanggil `fork()` untuk membuat proses, `exec()` untuk memuat program, lalu menunggu atau lanjut. Exit code 0 = sukses, > 0 = kode error. Arduino sebaliknya: single-tasking, tanpa OS, satu ruang memori.

System program dan background service

- Kategori: file management, status information (termasuk *registry*), file modification (editor, search), programming-language support (compiler, debugger), program loading and execution, communications, background services, application programs.
- Background service = **daemon**: jalan sejak boot, di *user context* (bukan kernel), mis. error logging, printing, process scheduling.
- Pandangan kebanyakan user terhadap OS dibentuk oleh system program, bukan system call.

Linker dan loader

1. Compiler menghasilkan **relocatable object file** (`.o`) yang bisa ditaruh di alamat mana pun.
2. **Linker** menggabungkan object file + library menjadi satu executable.
3. **Loader** memuat executable dari disk ke memori; *relocation* memberi alamat final.
4. Library dinamis (DLL di Windows, `.so` di Linux) dimuat saat dibutuhkan dan dipakai bersama oleh semua program.

- Aplikasi biasanya **OS-specific** karena system call dan format file tiap OS berbeda. Solusi lintas OS: bahasa interpreted (Python), bahasa ber-VM (Java), atau kompilasi ulang per OS (C).
- **ABI** (*Application Binary Interface*) = padanan API di level arsitektur: aturan antarmuka kode biner untuk OS + CPU tertentu.

Desain dan implementasi

- **User goals**: mudah dipakai, mudah dipelajari, andal, aman, cepat. **System goals**: mudah didesain, diimplementasi, dirawat; fleksibel, bebas error, efisien.

- **Policy vs mechanism**: policy = *apa* yang dilakukan (interrupt tiap 100 detik); mechanism = *bagaimana* (timer). Dipisah agar policy bisa diganti (100 → 200) tanpa mengubah mekanisme.
- Implementasi: level terendah assembly, badan utama C, system program C/C++/script. Bahasa tingkat tinggi lebih mudah di-port tapi lebih lambat.

Struktur kernel

Struktur	Ciri	Contoh	Plus	Minus
Simple	tanpa pemisahan jelas	MS-DOS	kecil	tidak terlindungi
Monolithic	semua fungsi kernel dalam satu ruang alamat, di bawah system-call interface	UNIX asli	cepat, overhead kecil	sulit dirawat dan diperluas
Layered	layer 0 = hardware, layer N = UI; tiap layer hanya memakai layer di bawahnya	konsep/abstraksi	mudah di-debug	sulit menentukan layer, lambat
Microkernel	sebanyak mungkin dipindah ke user space, komunikasi lewat message passing	Mach, Darwin (sebagian)	mudah diperluas dan di-port, lebih andal dan aman	overhead komunikasi user–kernel
Loadable kernel modules	komponen inti terpisah, dimuat saat perlu, bicara lewat interface	Linux, Solaris	fleksibel tanpa rebuild kernel	—
Hybrid	gabungan beberapa model	Linux (monolithic + modular), Windows (monolithic + microkernel), macOS	seimbang performa dan modularitas	kompleks

- **macOS/iOS**: Aqua UI + Cocoa, di bawahnya Darwin = Mach microkernel + BSD UNIX + I/O kit + kernel extensions.
- **Android**: kernel Linux yang dimodifikasi (+ power management), runtime dengan library inti dan Dalvik VM (sekarang ART); aplikasi ditulis Java + Android API.

Build dan boot

Membangun kernel Linux:

1. Unduh source dari kernel.org.
2. `make menuconfig` — konfigurasi.
3. `make` — hasilkan image kernel `vmlinuz`.
4. `make modules` lalu `make modules_install`.
5. `make install` — pasang kernel baru.

Urutan boot:

1. Daya menyala, eksekusi mulai di lokasi memori tetap.
2. **Bootstrap loader** di ROM/EEPROM (BIOS, atau UEFI di sistem modern) mencari kernel. Kadang dua tahap: boot block memuat loader dari disk.
3. Loader seperti **GRUB** memilih kernel, versi, dan opsi (mis. single-user mode).
4. Kernel dimuat ke memori dan dijalankan.

Debugging dan performa

- Error dicatat di **log file**. Aplikasi crash → **core dump** (memori proses). Kernel crash → **crash dump** (memori kernel).
- Performance tuning = menghilangkan bottleneck; alat: `top`, Task Manager.
- **Profiling** = sampling instruction pointer berkala untuk melihat tren statistik.
- Tracing: `strace` (system call sebuah proses), `gdb` (debugger source-level), `perf` (performa Linux), `tcpdump` (paket jaringan), BCC/eBPF dan DTrace (tracing user + kernel).
- *Kernighan's Law*: debugging dua kali lebih sulit daripada menulis kode, jadi jangan menulis kode terlalu "pintar".

BAB 3

03 Proses

INTISARI

Proses = program yang sedang dieksekusi. Program adalah file pasif di disk; ia menjadi proses saat dimuat ke memori. Satu program bisa menjadi banyak proses (mis. beberapa user menjalankan program yang sama).

Isi proses di memori

Dari alamat rendah ke tinggi:

Bagian	Isi
Text	kode program
Data	variabel global (initialized dan uninitialized)
Heap	memori dinamis saat runtime (`malloc`), tumbuh ke atas
Stack	parameter fungsi, return address, variabel lokal; tumbuh ke bawah

Selain itu ada *current activity*: program counter dan isi register.

State proses

Lima state: **new** (sedang dibuat), **ready** (menunggu CPU), **running** (sedang dieksekusi), **waiting** (menunggu event, mis. I/O), **terminated** (selesai).

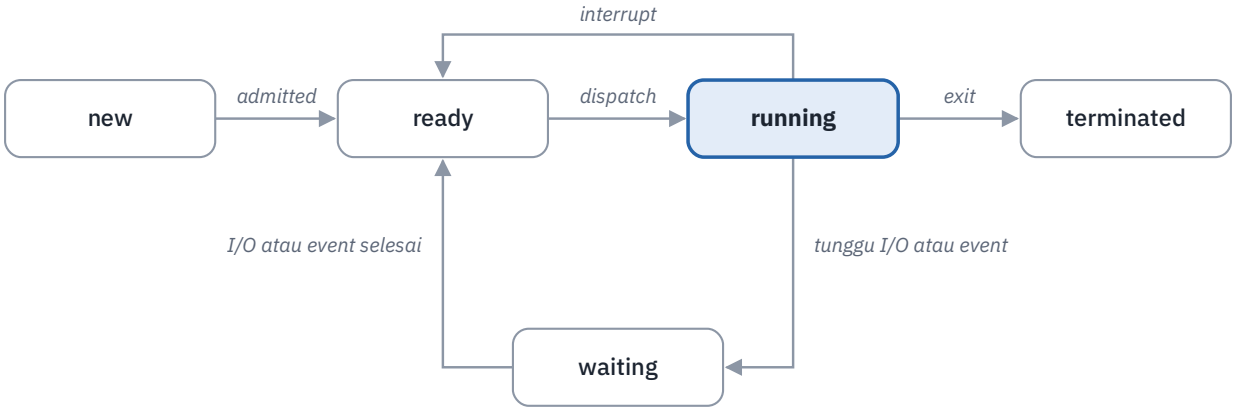


Diagram state proses · 5 state, 6 transisi

Interrupt (mis. quantum habis) mengembalikan proses dari running ke ready. Menunggu I/O memindahkannya ke waiting, dan setelah I/O selesai ia antre lagi di ready — tidak pernah langsung ke running.

Process Control Block (PCB)

PCB (*task control block*) menyimpan semua info satu proses; di Linux berupa `struct task_struct`.

- Process state, program counter, CPU registers
- CPU-scheduling info (prioritas, pointer antrean)

- Memory-management info (memori yang dialokasikan)
- Accounting info (waktu CPU terpakai, batas waktu)
- I/O status info (device yang dialokasikan, daftar file terbuka)

Scheduling dan context switch

- **Ready queue** = proses di memori yang siap jalan. **Wait queue** = proses yang menunggu event. Proses berpindah-pindah antar antrean.
- **Context switch**: simpan state proses lama ke PCB-nya, muat state proses baru dari PCB-nya. Waktu context switch adalah **overhead murni**; makin kompleks OS/PCB, makin lama. Hardware dengan banyak set register mempercepatnya.
- Mobile: iOS awal hanya satu proses foreground + background terbatas. Android menjalankan foreground dan background dengan **service** (tanpa UI, memori kecil).

Membuat dan mengakhiri proses

- Parent membuat child, membentuk **pohon proses**; tiap proses punya **pid**. Di Linux akar pohonnya `systemd` (pid 1).
- Opsi berbagi resource: semua, sebagian, atau tidak sama sekali. Opsi eksekusi: parent dan child jalan bersamaan, atau parent menunggu child selesai.
- Address space child: duplikat parent, atau dimuati program baru.
- UNIX: `fork()` membuat proses baru, `exec()` mengganti isi memori proses dengan program baru, `wait()` membuat parent menunggu child.

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();
    if (pid < 0) {            /* gagal */
        fprintf(stderr, "Fork Failed");
        return 1;
    } else if (pid == 0) {    /* child */
        execlp("/bin/ls", "ls", NULL);
    } else {                  /* parent */
        wait(NULL);
        printf("Child Complete\n");
    }
    return 0;
}
```

- Proses selesai dengan `exit()`; status dikembalikan ke parent lewat `pid = wait(&status);`, lalu resource-nya dibebaskan OS.
- Parent bisa menghentikan child dengan `abort()` jika child melebihi resource, tugasnya tak diperlukan lagi, atau parent akan keluar.
- **Cascading termination**: OS tertentu ikut mematikan semua anak-cucu jika parent mati.
- **Zombie** = child sudah selesai tapi parent belum memanggil `wait()`. **Orphan** = parent mati tanpa `wait()`.
- Urutan kepentingan proses Android, dari yang paling dipertahankan: foreground → visible → service → background → empty. Saat memori kurang, yang paling tidak penting dibuang duluan.

- Chrome multiproses: browser process (UI, I/O), renderer process per situs (di *sandbox*), plug-in process per jenis plug-in. Satu tab crash tidak menjatuhkan seluruh browser.

IPC (Interprocess Communication)

Proses bisa *independent* atau *cooperating*. Alasan bekerja sama: berbagi informasi, mempercepat komputasi, modularitas, kenyamanan.

	Shared memory	Message passing
Cara kerja	area memori dipakai bersama	`send(msg)` / `receive(msg)` lewat kernel
Kontrol	oleh proses user	oleh OS
Kecepatan	cepat (tanpa system call setelah setup)	lebih lambat (system call tiap pesan)
Masalah utama	sinkronisasi harus diatur sendiri	desain link
Cocok untuk	data besar di satu mesin	data kecil, sistem terdistribusi

Producer–consumer: *unbounded buffer* (producer tak pernah menunggu) vs *bounded buffer* (producer menunggu jika penuh, consumer menunggu jika kosong).

Bounded buffer dengan circular array (`in` , `out`) hanya bisa memakai **BUFFER_SIZE – 1** slot:

```
/* producer */
while (((in + 1) % BUFFER_SIZE) == out) ;  /* penuh, tunggu */
buffer[in] = next_produced;
in = (in + 1) % BUFFER_SIZE;

/* consumer */
while (in == out) ;                        /* kosong, tunggu */
next_consumed = buffer[out];
out = (out + 1) % BUFFER_SIZE;
```

Versi dengan `counter` bisa memakai semua slot, tapi `counter++` dan `counter--` tidak atomik (load → ubah → store). Interleaving yang salah menghasilkan **race condition**: dari counter = 5, hasilnya bisa 4 atau 6, bukan 5. Solusinya di Bab 6.

Detail message passing

- **Direct**: `send(P, msg)` , `receive(Q, msg)` . Link otomatis, tepat satu link per pasangan proses.
- **Indirect**: lewat **mailbox/port** `send(A, msg)` , `receive(A, msg)` . Satu link bisa melibatkan banyak proses. Jika P1 kirim dan P2, P3 sama-sama receive: batasi link ke dua proses, batasi satu receiver sekaligus, atau sistem memilih receiver dan memberi tahu sender.
- **Blocking (synchronous)**: sender ditahan sampai pesan diterima; receiver ditahan sampai ada pesan. Keduanya blocking = **rendezvous**.
- **Non-blocking (asynchronous)**: sender langsung lanjut; receiver dapat pesan valid atau null.
- **Buffering**: zero capacity (sender harus menunggu receiver), bounded capacity n (menunggu jika penuh), unbounded (tak pernah menunggu).

Contoh sistem IPC

- **POSIX shared memory**: `shm_open(name, O_CREAT | O_RDWR, 0666)` → `ftruncate(fd, 4096)` → `mmap()` ; baca/tulis lewat pointer hasil `mmap` .

- **Mach**: semua komunikasi berupa pesan, bahkan system call. Tiap task punya port Kernel dan Notify; `mach_msg()`, `mach_port_allocate()`.
- **Windows ALPC**: client membuka handle ke connection port, server membuat dua port privat dan memberikan satu ke client.
- **Ordinary pipe** (Windows: *anonymous pipe*): searah, butuh hubungan parent–child. `pipe(fd)` → `fd[0]` ujung baca, `fd[1]` ujung tulis.
- **Named pipe** (FIFO): dua arah, tanpa hubungan parent–child, bisa banyak proses.

Komunikasi client–server

- **Socket** = endpoint komunikasi = IP + port, mis. `161.25.19.8:1625`. Port < 1024 = *well-known*. `127.0.0.1` = loopback (mesin sendiri). Jenis di Java: TCP (connection-oriented), UDP (connectionless), multicast.
- **RPC** = memanggil prosedur di mesin lain seolah lokal. **Stub** client mencari server dan melakukan *marshalling* parameter; stub server membongkarnya lalu menjalankan prosedur.
- Beda arsitektur (big-endian vs little-endian) diatasi dengan format **XDR**. RPC idealnya menjamin pesan dieksekusi *exactly once*; OS menyediakan *matchmaker* untuk mempertemukan client dan server.

BAB 4

04 Thread dan Concurrency

INTISARI

Thread = unit eksekusi di dalam proses. Thread dalam satu proses berbagi code, data, dan file, tapi masing-masing punya **thread ID, program counter, register set, dan stack** sendiri. Membuat thread jauh lebih ringan daripada membuat proses.

Manfaat multithreading

- **Responsiveness** — bagian lain tetap jalan saat satu bagian blocked (penting untuk UI).
- **Resource sharing** — thread otomatis berbagi memori proses, lebih mudah daripada shared memory/message passing.
- **Economy** — membuat dan berganti thread lebih murah daripada proses.
- **Scalability** — bisa memanfaatkan banyak core.

Concurrency vs parallelism

- **Concurrency**: beberapa tugas sama-sama *mengalami kemajuan*; bisa di satu core lewat pergantian cepat.
- **Parallelism**: beberapa tugas *benar-benar jalan bersamaan*; butuh multicore.
- **Data parallelism**: data dibagi ke tiap core, operasi sama. **Task parallelism**: tiap thread mengerjakan operasi berbeda.
- Tantangan multicore: membagi aktivitas, keseimbangan beban, pemecahan data, ketergantungan data, testing/debugging.

Amdahl's Law

S = porsi serial, N = jumlah core. Porsi serial membatasi speedup secara tidak proporsional.

$$\text{speedup} \leq \frac{1}{S + \frac{1-S}{N}} \qquad \lim_{N \to \infty} \text{speedup} = \frac{1}{S}$$

Contoh S = 0,25 (75% paralel): N = 2 → 1/(0,25 + 0,375) = **1,6×**; N = 4 → 1/(0,25 + 0,1875) ≈ **2,29×**; N → ∞ → **4×**.

User thread, kernel thread, dan model pemetaan

User thread dikelola library di user space (Pthreads, Windows threads, Java threads). **Kernel thread** dikelola langsung oleh OS (Windows, Linux, macOS, iOS, Android).

Model	Pemetaan	Ciri	Contoh
Many-to-one	banyak user → 1 kernel thread	satu thread blocking = semua blocking; tidak bisa paralel di multicore	Solaris Green Threads, GNU Portable Threads
One-to-one	1 user → 1 kernel thread	concurrency tinggi; jumlah thread kadang dibatasi karena overhead	Windows, Linux
Many-to-many	banyak user → banyak kernel thread	OS membuat kernel thread secukupnya; jarang dipakai	Windows ThreadFiber
Two-level	seperti M:M + boleh *bind* user thread ke satu kernel thread	fleksibel	—

Pthreads (POSIX, IEEE 1003.1c)

Pthreads adalah **spesifikasi** API, bukan implementasi; umum di Linux dan macOS.

```c
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

int sum;                        /* dibagi antar-thread */
void *runner(void *param);

int main(int argc, char *argv[]) {
    pthread_t tid;
    pthread_attr_t attr;
    pthread_attr_init(&attr);                       /* atribut default */
    pthread_create(&tid, &attr, runner, argv[1]);   /* buat thread */
    pthread_join(tid, NULL);                        /* tunggu selesai */
    printf("sum = %d\n", sum);
    return 0;
}

void *runner(void *param) {
    int upper = atoi(param);
    sum = 0;
    for (int i = 1; i <= upper; i++) sum += i;
    pthread_exit(0);
}
```

- Kompilasi: `gcc prog.c -o prog -pthread`.
- Join banyak thread: `for (int i = 0; i < 10; i++) pthread_join(workers[i], NULL);`
- Windows: `CreateThread()` lalu `WaitForSingleObject()`. Java: implementasikan `Runnable` (praktik standar) atau extend `Thread`; `start()` lalu `join()`.

Implicit threading

Pembuatan dan pengelolaan thread diserahkan ke compiler/runtime, bukan programmer.

- **Thread pool**: thread dibuat di awal dan menunggu tugas. Lebih cepat daripada membuat thread baru, jumlah thread terbatas ukuran pool, tugas bisa dijadwalkan. Java: `Executors.newSingleThreadExecutor()`, `newFixedThreadPool(n)`, `newCachedThreadPool()`.
- **Fork-join**: tugas dipecah rekursif lalu hasilnya digabung. Java: `ForkJoinPool`, `RecursiveTask` (mengembalikan hasil lewat `compute()`) dan `RecursiveAction` (tanpa hasil).

```
Task(problem):
    if problem cukup kecil: selesaikan langsung
    else:
        sub1 = fork(Task(bagian 1)); sub2 = fork(Task(bagian 2))
        return gabung(join(sub1), join(sub2))
```

- **OpenMP** (C, C++, Fortran): `#pragma omp parallel` membuat thread sebanyak core; `#pragma omp parallel for` memparalelkan loop. Kompilasi dengan `-fopenmp`.
- **Grand Central Dispatch** (Apple): block `^{ ... }` atau closure Swift dimasukkan ke dispatch queue. *Serial queue* (FIFO, satu per satu, mis. main queue) vs *concurrent queue* (FIFO, beberapa sekaligus; QoS: user-interactive, user-initiated, utility, background).
- **Intel TBB**: template library C++, mis. `parallel_for`.

Threading issues

- **`fork()` di proses multithread**: menduplikasi semua thread atau hanya thread pemanggil? Beberapa UNIX punya dua versi. Jika langsung `exec()`, cukup duplikasi thread pemanggil karena seluruh proses akan diganti.
- **Signal**: dibangkitkan event → dikirim ke proses → ditangani *default handler* (kernel) atau *user-defined handler*. Di proses multithread, signal bisa dikirim ke thread yang bersangkutan, ke semua thread, ke thread tertentu, atau ke satu thread khusus penerima signal.
- **Thread cancellation**: *asynchronous* (target langsung dihentikan) vs *deferred* (default; target mengecek sendiri di *cancellation point* seperti `pthread_testcancel()`, lalu cleanup handler dipanggil). Pthreads: `pthread_cancel(tid)`. Java: `interrupt()` lalu `isInterrupted()`.
- **Thread-local storage (TLS)**: tiap thread punya salinan data sendiri yang bertahan lintas pemanggilan fungsi (beda dengan variabel lokal). Berguna di thread pool.
- **Scheduler activations**: M:M dan two-level memakai **LWP** (*lightweight process*) sebagai "prosesor virtual" yang terikat ke kernel thread. Kernel memberi tahu library thread lewat **upcall** agar jumlah kernel thread tetap pas.

Contoh OS

- **Windows**: one-to-one. Context thread = register set + stack user/kernel + private storage. Struktur: ETHREAD dan KTHREAD (kernel space), TEB (user space; thread id, user stack, TLS).
- **Linux**: tidak membedakan proses dan thread; keduanya disebut **task**. Thread dibuat dengan `clone()`, flag menentukan apa yang dibagi (mis. `CLONE_VM` untuk address space). Semua direpresentasikan `struct task_struct`.

05 BAB 5

CPU Scheduling

INTISARI

CPU scheduler memilih proses dari ready queue untuk diberi CPU. SJF memberi average waiting time minimum, RR memberi response time terbaik, dan OS modern memakai varian priority + feedback queue yang preemptive.

Konsep dasar

- Eksekusi proses = siklus **CPU burst** dan **I/O burst**. Kebanyakan burst pendek, sedikit yang panjang.
- Keputusan scheduling terjadi saat proses: (1) running → waiting, (2) running → ready, (3) waiting → ready, (4) terminate.
- Hanya di (1) dan (4) = **nonpreemptive** (proses memegang CPU sampai selesai atau menunggu). Bisa juga di (2) dan (3) = **preemptive**. Windows, macOS, Linux, UNIX semuanya preemptive.
- Preemption bisa memicu race condition pada data bersama (Bab 6).
- **Dispatcher** memberi CPU ke proses terpilih: context switch → pindah ke user mode → lompat ke lokasi yang benar. **Dispatch latency** = waktu menghentikan satu proses dan memulai yang lain.

Kriteria dan rumus

Kriteria	Arti	Target
CPU utilization	seberapa sibuk CPU	max
Throughput	jumlah proses selesai per satuan waktu	max
Turnaround time	waktu dari datang sampai selesai	min
Waiting time	total waktu di ready queue	min
Response time	waktu dari datang sampai pertama kali dilayani	min

```
Turnaround = Completion - Arrival
Waiting    = Turnaround - Burst
Response   = First run  - Arrival
```

Ringkasan algoritma

Algoritma	Preemptive?	Cara pilih	Plus	Minus
FCFS	tidak	urutan datang	sederhana	**convoy effect** (proses pendek menunggu proses panjang)
SJF	tidak	burst berikutnya terpendek	**optimal** untuk average waiting time	panjang burst harus ditebak
SRTF	ya	sisa burst terpendek, dicek ulang tiap ada proses datang	waiting time lebih kecil lagi	butuh prediksi; starvation proses panjang

Algoritma	Preemptive?	Cara pilih	Plus	Minus
Round Robin	ya	bergiliran, jatah waktu q	response time bagus, adil	turnaround biasanya lebih besar dari SJF
Priority	keduanya	prioritas tertinggi (angka **kecil** = prioritas tinggi)	fleksibel	**starvation** → solusi **aging**
Priority + RR	ya	prioritas tertinggi; prioritas sama → RR	gabungan keduanya	—
Multilevel queue	tergantung	antrean per prioritas/jenis proses, proses tetap di antreannya	memisahkan interaktif vs batch	kaku
Multilevel feedback queue	ya	proses bisa naik/turun antrean	paling fleksibel, bisa aging	paling kompleks parameternya

Contoh hitung dengan Gantt chart

Semua proses tiba di t = 0, kecuali SRTF yang memakai arrival time. Angka di bawah batang = waktu mulai/selesai.

FCFS — urutan datang P1, P2, P3

`burst P1 = 24, P2 = 3, P3 = 3`

`avg waiting = (0 + 24 + 27) / 3 = 17`



FCFS — urutan datang P2, P3, P1

`burst sama, urutan dibalik`

`avg waiting = (6 + 0 + 3) / 3 = 3`



Urutan kedatangan saja sudah mengubah rata-rata dari 17 menjadi 3: itulah *convoy effect*.

SJF (nonpreemptive)

`burst P1 = 6, P2 = 8, P3 = 7, P4 = 3`

`avg waiting = (3 + 16 + 9 + 0) / 4 = 7`



SRTF (SJF preemptive)

`(arrival, burst): P1(0, 8) P2(1, 4) P3(2, 9) P4(3, 5)`

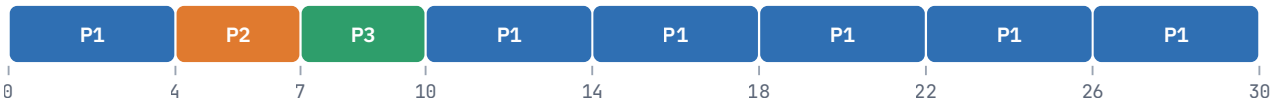
`avg waiting = [(10-1) + (1-1) + (17-2) + (5-3)] / 4 = 6,5`



Round Robin, q = 4

`burst P1 = 24, P2 = 3, P3 = 3`

`avg waiting = (6 + 4 + 7) / 3 ≈ 5,67`



Priority (angka kecil = prioritas tinggi)

`(burst, prio): P1(10, 3) P2(1, 1) P3(2, 4) P4(1, 5) P5(5, 2)`

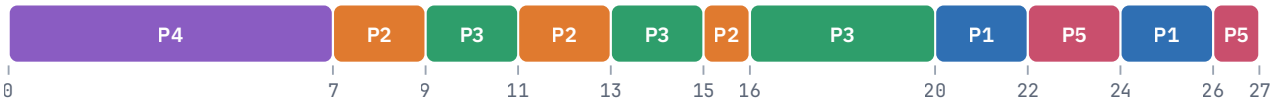
`avg waiting = (6 + 0 + 16 + 18 + 1) / 5 = 8,2`



Priority + Round Robin, q = 2

`(burst, prio): P1(4, 3) P2(5, 2) P3(8, 2) P4(7, 1) P5(3, 3)`

`avg waiting = (22 + 11 + 12 + 0 + 24) / 5 = 13,8`



Prediksi burst untuk SJF

Panjang burst berikutnya ditebak dengan *exponential averaging*. t_n = panjang burst ke-n yang sebenarnya, τ_n = prediksi sebelumnya, $0 \le \alpha \le 1$ (biasanya ½).

$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n$$

$\alpha = 0$ → riwayat terbaru diabaikan ($\tau_{n+1} = \tau_n$). $\alpha = 1$ → hanya burst terakhir yang dihitung ($\tau_{n+1} = t_n$). Jika diuraikan, bobot burst lama makin kecil secara eksponensial.

Round Robin: memilih q

- Dengan n proses dan quantum q, tiap proses dapat 1/n waktu CPU, dan tak ada yang menunggu lebih dari **(n − 1)·q**.
- q besar → RR menjadi FCFS. q kecil → terlalu banyak context switch.
- Aturan praktis: q = 10–100 ms, context switch < 10 µs, dan **80% CPU burst sebaiknya lebih pendek dari q**.

Multilevel feedback queue

Contoh dari slide:

1. Proses baru masuk **Q0** (RR, q = 8 ms).
2. Belum selesai dalam 8 ms → turun ke **Q1** (RR, q = 16 ms).
3. Masih belum selesai → turun ke **Q2** (FCFS).

Parameter MLFQ: jumlah antrean, algoritma tiap antrean, kapan proses dinaikkan, kapan diturunkan, dan antrean mana yang dimasuki proses baru.

Thread scheduling

- **PCS** (*process-contention scope*): library memilih user thread mana yang jalan di LWP; persaingan di dalam satu proses (model M:1 dan M:M).
- **SCS** (*system-contention scope*): kernel memilih kernel thread mana yang dapat CPU; persaingan antar semua thread di sistem.
- Pthreads: `pthread_attr_setscope(&attr, PTHREAD_SCOPE_SYSTEM)` atau `PTHREAD_SCOPE_PROCESS`. Linux dan macOS hanya mengizinkan `PTHREAD_SCOPE_SYSTEM`.

Multiprocessor scheduling

- Arsitektur: multicore, multithreaded core, NUMA, heterogeneous (mis. big.LITTLE).
- **SMP**: tiap prosesor menjadwalkan dirinya sendiri; ready queue bersama atau per-prosesor.
- **Chip multithreading (CMT)** / hyperthreading: tiap core punya beberapa hardware thread; saat satu thread kena *memory stall*, core pindah ke thread lain. Quad-core × 2 thread = 8 logical CPU. Ada dua level scheduling: OS memilih software thread per logical CPU, core memilih hardware thread.
- **Load balancing**: *push migration* (task periodik memindahkan beban dari CPU sibuk) dan *pull migration* (CPU idle menarik tugas).
- **Processor affinity**: thread "betah" di CPU yang cache-nya sudah berisi datanya. *Soft affinity* = diusahakan; *hard affinity* = proses menentukan sendiri himpunan CPU-nya. Load balancing bisa merusak affinity.
- OS yang **NUMA-aware** mengalokasikan memori yang dekat dengan CPU tempat thread berjalan.

Real-time scheduling

- **Soft real-time**: tugas kritis diberi prioritas tertinggi, tanpa jaminan waktu. **Hard real-time**: tugas wajib selesai sebelum deadline.
- **Event latency** = interrupt latency (interrupt datang → ISR mulai) + dispatch latency. Fase konflik dispatch: preempt proses di kernel mode, lalu proses prioritas rendah melepas resource yang dibutuhkan proses prioritas tinggi.
- Proses periodik punya waktu proses t, deadline d, periode p dengan $0 \le t \le d \le p$; rate = 1/p.
- **Rate Monotonic (RM)**: prioritas statis, periode lebih pendek = prioritas lebih tinggi. Contoh gagal: P1 (p = 50, t = 25), P2 (p = 80, t = 35); utilisasi 0,5 + 0,4375 ≈ 94%, P2 melewati deadline di t = 80. Batas aman RM dari buku: utilisasi $\le N(2^{1/N} - 1)$, ≈ 83% untuk N = 2.
- **Earliest Deadline First (EDF)**: prioritas dinamis, deadline terdekat = prioritas tertinggi. Kasus P1/P2 di atas lolos dengan EDF; secara teori bisa sampai utilisasi 100%.
- **Proportional share**: total T share dibagi; aplikasi dengan N share dijamin N/T waktu prosesor.
- POSIX.1b: `SCHED_FIFO` (FCFS, tanpa time slice untuk prioritas sama) dan `SCHED_RR` (dengan time slice); atur lewat `pthread_attr_setschedpolicy()`.

Contoh OS

- **Linux ≤ 2.5**: scheduler O(1), prioritas real-time 0–99 dan nice 100–140, dua array *active* dan *expired*. Respons interaktif buruk.
- **Linux ≥ 2.6.23: CFS** (*Completely Fair Scheduler*). Nice −20 s.d. +19 (lebih kecil = prioritas lebih tinggi). Tiap task punya `vruntime`; yang dipilih adalah task dengan **vruntime terkecil** (disimpan di red-black tree). *Target latency* = selang waktu tiap task harus jalan minimal sekali. Prioritas global: real-time 0–99, nice −20 → 100, nice +19 → 139. NUMA-aware lewat *scheduling domain*.

- **Windows**: preemptive berbasis prioritas, 32 level. Variable class 1–15, real-time class 16–31, prioritas 0 untuk thread memory-management, idle thread jika kosong. Prioritas = priority class (REALTIME, HIGH, ABOVE_NORMAL, NORMAL, BELOW_NORMAL, IDLE) + relative priority (TIME_CRITICAL … IDLE). Quantum habis → prioritas turun (tak di bawah base); selesai menunggu → dinaikkan; window foreground mendapat boost 3×. Windows 7 menambah *user-mode scheduling* (UMS).
- **Solaris**: enam kelas (TS default, IA, RT, SYS, FSS, FP); TS memakai MLFQ dengan *dispatch table* yang bisa diatur admin; prioritas sama → RR.

Evaluasi algoritma

- **Deterministic modeling**: hitung dengan beban kerja tertentu. Contoh buku, lima proses burst 10, 29, 3, 7, 12: FCFS 28 ms, SJF 13 ms, RR (q = 10) 23 ms. Cepat, tapi hanya berlaku untuk input itu.
- **Queueing model**: kedatangan dan burst dimodelkan probabilistik. **Little's formula** $n = \lambda \times W$ (panjang antrean = laju kedatangan × waktu tunggu). Contoh: 7 proses/detik, 14 proses di antrean → W = 2 detik.
- **Simulasi**: model sistem yang diprogram; data dari random generator atau *trace tape* kejadian nyata. Lebih akurat, lebih mahal.
- **Implementasi langsung**: paling akurat, tapi biaya dan risiko tinggi.

BAB 6

06 Synchronization Tools

INTISARI

Akses bersamaan ke data bersama bisa membuat data tidak konsisten (**race condition**: hasil tergantung urutan eksekusi). Solusinya melindungi **critical section** dengan alat dari level hardware (memory barrier, test_and_set, CAS) sampai level tinggi (mutex, semaphore, monitor).

Contoh race condition di kernel: P0 dan P1 sama-sama memanggil `fork()` dan membaca `next_available_pid` ; tanpa proteksi, dua proses bisa mendapat pid yang sama.

Critical-section problem

Struktur tiap proses: **entry section** (minta izin) → **critical section** → **exit section** → **remainder section**.

Solusi wajib memenuhi tiga syarat:

1. **Mutual exclusion** — jika Pi di critical section, proses lain tidak boleh masuk critical section-nya.
2. **Progress** — jika tak ada yang di critical section dan ada yang ingin masuk, pemilihan siapa yang masuk tidak boleh ditunda tanpa batas (yang di remainder section tidak ikut menentukan).
3. **Bounded waiting** — ada batas berapa kali proses lain boleh masuk setelah Pi meminta, sebelum Pi diizinkan.

Asumsi: tiap proses berjalan dengan kecepatan > 0, tanpa asumsi kecepatan relatif.

Solusi awal dan kenapa kurang

- **Disable interrupt** di entry, enable di exit: critical section yang lama membekukan sistem, bisa starvation, dan tidak bekerja di multiprocessor.
- **Software Solution 1** (hanya `int turn`): `while (turn == j); /* CS */ turn = j;` — mutual exclusion terpenuhi, tapi **progress gagal** karena proses dipaksa bergantian ketat.

Peterson's Solution (2 proses)

Variabel bersama: `int turn` (giliran siapa) dan `boolean flag[2]` (`flag[i] = true` berarti Pi siap masuk).

```
while (true) {
    flag[i] = true;
    turn = j;                       /* persilakan yang lain dulu */
    while (flag[j] && turn == j) ;  /* tunggu */
    /* critical section */
    flag[i] = false;
    /* remainder section */
}
```

- Memenuhi ketiga syarat: Pi masuk hanya jika `flag[j] == false` atau `turn == i`.
- **Tidak dijamin benar di arsitektur modern** karena prosesor/compiler boleh **menyusun ulang (reorder)** instruksi yang tidak saling bergantung, sehingga dua proses bisa masuk bersamaan.

Memory model dan memory barrier

- **Strongly ordered**: perubahan memori satu prosesor langsung terlihat oleh semua. **Weakly ordered**: belum tentu langsung terlihat.
- **Memory barrier** memaksa semua load/store sebelumnya selesai dan terlihat sebelum load/store berikutnya.
- Contoh: Thread 2 menjalankan `x = 100; flag = true;`, Thread 1 menunggu `flag` lalu mencetak `x`. Tanpa barrier, reorder bisa membuat output **0**. Dengan `memory_barrier()` di antara kedua instruksi, output dijamin **100**.

Instruksi atomik hardware

```
bool test_and_set(bool *target) {   /* atomik */
    bool rv = *target;
    *target = true;
    return rv;                      /* nilai lama */
}

int compare_and_swap(int *value, int expected, int new_value) {  /* atomik */
    int temp = *value;
    if (*value == expected) *value = new_value;
    return temp;                    /* nilai lama */
}
```

- Lock dengan TAS: `while (test_and_set(&lock)); /* CS */ lock = false;` (lock awal false).
- Lock dengan CAS: `while (compare_and_swap(&lock, 0, 1) != 0); /* CS */ lock = 0;` (lock awal 0).
- Keduanya memenuhi mutual exclusion dan progress, tapi **tidak bounded waiting**. Versi CAS dengan array `waiting[n]` memperbaikinya: saat keluar, proses mencari j berikutnya secara melingkar yang sedang menunggu dan menyerahkan giliran langsung, sehingga tiap proses menunggu paling lama **n − 1** giliran.
- **Atomic variable**: tipe dasar dengan update atomik, dibangun di atas CAS:

```
void increment(atomic_int *v) {
    int temp;
    do { temp = *v; } while (temp != compare_and_swap(v, temp, temp + 1));
}
```

Mutex lock

- Alat paling sederhana: variabel boolean `available`. `acquire()` sebelum critical section, `release()` sesudahnya; keduanya harus atomik (biasanya pakai CAS).
- Implementasi dengan busy waiting disebut **spinlock**. Boros CPU, tapi bagus bila lock dipegang sangat singkat di sistem multicore karena tidak butuh context switch.

Semaphore

Semaphore S = integer yang hanya diakses lewat dua operasi atomik: `wait()` (P) dan `signal()` (V).

```
wait(S)   { while (S <= 0) ; S--; }   /* versi busy waiting */
signal(S) { S++; }
```

- **Counting semaphore**: nilai bebas, untuk resource dengan beberapa instance. **Binary semaphore**: 0 atau 1, setara mutex.

- Pola critical section: `mutex = 1`, lalu `wait(mutex); CS; signal(mutex);`
- Pola urutan (S1 harus sebelum S2): `synch = 0`. P1: `S1; signal(synch);` — P2: `wait(synch); S2;`

Implementasi tanpa busy waiting memakai waiting queue, `block()` (masukkan pemanggil ke antrean semaphore) dan `wakeup(P)` (pindahkan P ke ready queue):

```
typedef struct { int value; struct process *list; } semaphore;

wait(semaphore *S) {
    S->value--;
    if (S->value < 0) { tambahkan proses ini ke S->list; block(); }
}
signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) { ambil P dari S->list; wakeup(P); }
}
```

Jika `value` negatif, besarnya = jumlah proses yang sedang menunggu.

> **CATATAN**
> Kesalahan umum: `signal` sebelum `wait` (mutual exclusion jebol), `wait` dua kali (deadlock), atau lupa salah satunya.

Monitor dan condition variable

- **Monitor** = tipe data abstrak tingkat tinggi; variabel internal hanya diakses prosedur di dalamnya, dan **hanya satu proses aktif di dalam monitor** pada satu waktu.
- **Condition variable** `condition x;` dengan dua operasi: `x.wait()` (tangguhkan pemanggil sampai ada `x.signal()`) dan `x.signal()` (bangunkan satu proses yang menunggu; jika tidak ada yang menunggu, **tidak berefek** — beda dengan semaphore yang menyimpan nilai).
- Implementasi dengan semaphore: `mutex = 1` membungkus tiap prosedur; `next = 0` dan `next_count` untuk proses yang menunggu di dalam monitor; tiap condition x punya `x_sem = 0` dan `x_count`.
- **Conditional wait** `x.wait(c)`: c = nomor prioritas; nilai **terkecil** dibangunkan duluan.

Contoh monitor alokasi satu resource (yang rencana pemakaiannya paling singkat dilayani dulu):

```
monitor ResourceAllocator {
    boolean busy;
    condition x;
    void acquire(int time) { if (busy) x.wait(time); busy = true; }
    void release()         { busy = false; x.signal(); }
    initialization_code()  { busy = false; }
}
/* pemakaian: R.acquire(t); ... akses resource ...; R.release(); */
```

Ringkasan alat

Alat	Level	Busy waiting?	Catatan
Disable interrupt	kernel, uniprocessor	—	tidak skalabel di multiprocessor
Peterson	software, 2 proses	ya	butuh memory barrier di CPU modern
test_and_set / CAS	hardware	ya	dasar alat lain; versi sederhana tidak bounded waiting
Atomic variable	hardware + library	ya (singkat)	untuk update satu variabel, mis. counter
Mutex (spinlock)	OS/library	ya	cocok untuk lock singkat di multicore
Semaphore	OS/library	tidak (versi antrean)	counting atau binary; rawan salah pakai
Monitor	bahasa pemrograman	tidak	paling aman dipakai; mutual exclusion otomatis

Evaluasi dari buku: pada contention rendah sampai sedang, CAS umumnya lebih cepat; pada contention tinggi, alat yang memblokir (mutex/semaphore dengan antrean) lebih baik.

Liveness

Liveness = sifat yang menjamin proses terus maju. Menunggu tanpa batas melanggar progress dan bounded waiting.

- **Deadlock**: dua proses atau lebih saling menunggu event yang hanya bisa dipicu oleh salah satu dari mereka. Contoh S = Q = 1: P0 `wait(S); wait(Q);`, P1 `wait(Q); wait(S);` — P0 memegang S, P1 memegang Q, keduanya macet. Detail di Bab 8.
- **Starvation**: proses tidak pernah dikeluarkan dari antrean semaphore (mis. antrean LIFO).
- **Priority inversion**: proses prioritas rendah memegang lock yang dibutuhkan proses prioritas tinggi, lalu didahului proses prioritas menengah. Solusi: **priority-inheritance protocol** (pemegang lock sementara mewarisi prioritas tertinggi yang menunggunya).

07 BAB 7

Contoh Sinkronisasi

INTISARI

Tiga masalah klasik dipakai untuk menguji skema sinkronisasi baru: bounded buffer, readers-writers, dan dining philosophers. Kuncinya ada di nilai awal semaphore dan urutan `wait()`.

Bounded buffer (n slot)

Semaphore: `mutex = 1` (akses buffer), `empty = n` (jumlah slot kosong), `full = 0` (jumlah slot terisi).

```
/* producer */
while (true) {
    /* produce item */
    wait(empty);
    wait(mutex);
    /* taruh item di buffer */
    signal(mutex);
    signal(full);
}
```

```
/* consumer */
while (true) {
    wait(full);
    wait(mutex);
    /* ambil item dari buffer */
    signal(mutex);
    signal(empty);
    /* consume item */
}
```

CATATAN

Urutan `wait(empty)` lalu `wait(mutex)` penting: jika dibalik, producer bisa memegang mutex sambil menunggu slot kosong, dan consumer tak bisa masuk → deadlock.

Jejak N = 2 dari slide:

Langkah	Kejadian	empty	full
0	buffer kosong	2	0
1	producer masukkan A	1	1
2	producer masukkan B	0	2
3	producer coba masukkan C; `wait(empty)` tertahan	0	2
4	consumer ambil A; producer boleh lanjut	1	1
5	producer masukkan C	0	2

Readers-writers

Banyak reader boleh membaca bersamaan; writer harus eksklusif. Data bersama: `rw_mutex = 1`, `mutex = 1` (melindungi `read_count`), `read_count = 0`.

```
/* writer */
wait(rw_mutex);
/* menulis */
signal(rw_mutex);

/* reader */
wait(mutex);
read_count++;
if (read_count == 1) wait(rw_mutex);   /* reader pertama mengunci writer */
signal(mutex);
/* membaca */
wait(mutex);
read_count--;
if (read_count == 0) signal(rw_mutex); /* reader terakhir membuka */
signal(mutex);
```

- Solusi di atas = **first readers-writers problem**: reader diutamakan, writer bisa **starvation**.
- **Second readers-writers problem**: begitu writer siap, reader baru tidak boleh masuk; reader yang bisa starvation.
- Banyak kernel menyediakan **reader-writer lock** langsung.

Dining philosophers (5 filsuf, 5 sumpit)

Solusi semaphore sederhana, `chopstick[5]` semua bernilai 1:

```
while (true) {
    wait(chopstick[i]);
    wait(chopstick[(i + 1) % 5]);
    /* makan */
    signal(chopstick[i]);
    signal(chopstick[(i + 1) % 5]);
    /* berpikir */
}
```

Masalahnya: jika kelima filsuf mengambil sumpit kiri bersamaan, semua menunggu sumpit kanan → **deadlock**. Perbaikan (dari buku): batasi maksimal 4 filsuf duduk; ambil sumpit hanya jika keduanya tersedia; atau asimetris (ganjil ambil kiri dulu, genap kanan dulu).

Solusi monitor: filsuf hanya makan jika kedua tetangganya tidak sedang makan. **Bebas deadlock, tapi starvation masih mungkin.**

```
monitor DiningPhilosophers {
    enum { THINKING, HUNGRY, EATING } state[5];
    condition self[5];

    void pickup(int i)  { state[i] = HUNGRY; test(i);
                          if (state[i] != EATING) self[i].wait(); }
    void putdown(int i) { state[i] = THINKING;
                          test((i + 4) % 5); test((i + 1) % 5); }  /* cek kiri & kanan */
    void test(int i) {
        if (state[(i + 4) % 5] != EATING && state[i] == HUNGRY &&
            state[(i + 1) % 5] != EATING) {
            state[i] = EATING; self[i].signal();
        }
    }
    initialization_code() { for (int i = 0; i < 5; i++) state[i] = THINKING; }
}
/* filsuf i: DiningPhilosophers.pickup(i); makan; DiningPhilosophers.putdown(i); */
```

Sinkronisasi di kernel

- **Windows**: uniprocessor memakai *interrupt mask*; multiprocessor memakai spinlock (thread pemegang spinlock tak pernah di-preempt). User space memakai **dispatcher object** (mutex, semaphore, event, timer) dengan state *signaled* (tersedia) atau *non-signaled* (thread akan blok). Event mirip condition variable.
- **Linux**: sebelum 2.6 menonaktifkan interrupt untuk critical section pendek; sejak 2.6 kernel **fully preemptive**. Menyediakan semaphore, atomic integer, spinlock, dan versi reader-writer-nya. Di single-CPU, spinlock diganti enable/disable kernel preemption.
- Atomic integer Linux: `atomic_t counter;` dengan `atomic_set(&counter, 5)`, `atomic_add(10, &counter)`, `atomic_sub(4, &counter)`, `atomic_inc(&counter)`, `value = atomic_read(&counter)`.

API POSIX (C, -pthread)

```
/* mutex */
pthread_mutex_t mutex;
pthread_mutex_init(&mutex, NULL);
pthread_mutex_lock(&mutex);   /* critical section */   pthread_mutex_unlock(&mutex);

/* named semaphore: bisa dipakai proses yang tidak berhubungan */
sem_t *sem = sem_open("SEM", O_CREAT, 0666, 1);
sem_wait(sem);   /* critical section */   sem_post(sem);

/* unnamed semaphore: antar-thread dalam satu proses */
sem_t s;
sem_init(&s, 0, 1);           /* 0 = tidak dibagi antar proses, nilai awal 1 */
sem_wait(&s);    /* critical section */   sem_post(&s);

/* condition variable: selalu dipasangkan dengan mutex */
pthread_cond_t cond_var;
pthread_cond_init(&cond_var, NULL);

pthread_mutex_lock(&mutex);               /* thread yang menunggu a == b */
while (a != b) pthread_cond_wait(&cond_var, &mutex);
pthread_mutex_unlock(&mutex);

pthread_mutex_lock(&mutex);               /* thread yang memberi sinyal */
a = b;
```

```
pthread_cond_signal(&cond_var);
pthread_mutex_unlock(&mutex);
```

> **CATATAN**
> Pakai `while`, bukan `if`, saat menunggu condition variable: setelah dibangunkan, kondisi harus dicek ulang.

Java

- **Monitor**: tiap objek punya satu lock. Method `synchronized` mewajibkan thread memegang lock objek; jika dipegang thread lain, thread masuk **entry set**.
- `wait()`: lepas lock → state blocked → masuk **wait set**. `notify()`: satu thread acak dari wait set dipindah ke entry set dan menjadi runnable, lalu bersaing lagi untuk lock. `notifyAll()` memindahkan semuanya.
- **ReentrantLock**: `lock()` lalu `unlock()` di blok `finally` agar lock tetap dilepas walau ada exception.
- **Semaphore**: `new Semaphore(1)`, `acquire()`, `release()`.
- **Condition**: `lock.newCondition()`, lalu `await()` dan `signal()`.

Pendekatan alternatif

- **Transactional memory**: `atomic { S }` menjalankan rangkaian baca/tulis memori secara atomik tanpa lock eksplisit.
- **OpenMP**: `#pragma omp critical { count += value; }` menjadikan blok itu critical section.
- **Bahasa fungsional** (Erlang, Scala): variabel immutable, tidak ada state yang berubah, sehingga data race tidak terjadi.

BAB 8

08 Deadlock

INTISARI

Deadlock terjadi jika empat kondisi berlaku **bersamaan**; mematahkan satu saja sudah cukup untuk mencegahnya. Ada empat sikap: prevention, avoidance (Banker), detection + recovery, atau mengabaikan (yang dipakai kebanyakan OS, termasuk Linux dan Windows).

Model sistem

- Resource type R1 … Rm (CPU cycle, memori, I/O device); tiap Ri punya Wi instance.
- Tiap thread memakai resource dengan urutan: **request → use → release**.
- Contoh dengan semaphore S1 = S2 = 1: T1 `wait(S1); wait(S2);`, T2 `wait(S2); wait(S1);`. Jika T1 dapat S1 dan T2 dapat S2, keduanya saling menunggu.

Empat kondisi perlu (Coffman)

1. **Mutual exclusion** — hanya satu thread yang bisa memakai resource pada satu waktu.
2. **Hold and wait** — thread memegang minimal satu resource sambil menunggu resource lain yang dipegang thread lain.
3. **No preemption** — resource hanya dilepas sukarela oleh pemegangnya setelah tugasnya selesai.
4. **Circular wait** — ada rantai {T0, …, Tn}: T0 menunggu T1, T1 menunggu T2, …, Tn menunggu T0.

Resource-allocation graph (RAG)

- Simpul: thread (lingkaran) dan resource type (kotak, titik di dalamnya = instance).
- **Request edge** Ti → Rj (thread meminta). **Assignment edge** Rj → Ti (instance sudah diberikan).

Kondisi graf	Kesimpulan
Tidak ada cycle	**pasti tidak** deadlock
Ada cycle, tiap resource satu instance	**pasti** deadlock
Ada cycle, resource banyak instance	**mungkin** deadlock

- Contoh slide 8.9: cycle T1 → R1 → T2 → R3 → T3 → R2 → T1 dan T2 → R3 → T3 → R2 → T2; tak ada yang bisa melepas → deadlock.
- Contoh slide 8.10: cycle T1 → R1 → T3 → R2 → T1, tapi T2 dan T4 bisa selesai dan melepas instance R1/R2 → **tidak deadlock**.

Deadlock prevention

Patahkan salah satu dari empat kondisi:

Kondisi	Cara mematahkan	Kekurangan
Mutual exclusion	tidak perlu untuk resource sharable (file read-only)	tidak bisa untuk resource non-sharable (mutex, printer)
Hold and wait	minta semua resource sebelum mulai, atau hanya boleh meminta saat tidak memegang apa pun	utilisasi resource rendah, starvation
No preemption	jika permintaan tak bisa dipenuhi segera, lepaskan semua yang dipegang; thread di-restart setelah bisa mendapat semuanya	hanya cocok untuk resource yang state-nya bisa disimpan (CPU register, memori)
Circular wait	beri nomor urut tiap resource, minta selalu dengan urutan **naik**	paling umum dipakai; urutan harus dipatuhi semua kode

Contoh circular wait: `first_mutex = 1`, `second_mutex = 5`. Semua thread wajib mengunci `first_mutex` dulu, lalu `second_mutex`; thread yang mengunci `second_mutex` duluan melanggar aturan.

Deadlock avoidance

Sistem butuh info *a priori*: tiap thread mendeklarasikan **jumlah maksimum** tiap resource yang mungkin dipakai. Algoritma memeriksa *resource-allocation state* (available, allocated, maximum demand) setiap ada request.

- **Safe state** = ada urutan ⟨T1, …, Tn⟩ dari **semua** thread sehingga kebutuhan sisa tiap Ti bisa dipenuhi oleh *available* + resource yang dipegang semua Tj dengan j < i.
- Safe → pasti tidak deadlock. Unsafe → **mungkin** deadlock (tidak pasti). Deadlock ⊂ unsafe. Avoidance = jangan pernah masuk unsafe state.
- Resource satu instance → **RAG + claim edge**. Resource banyak instance → **Banker's algorithm**.

RAG scheme: *claim edge* Ti ⇢ Rj (garis putus-putus) = Ti mungkin meminta Rj nanti. Claim → request saat diminta → assignment saat diberikan → kembali ke claim saat dilepas. Request **hanya dikabulkan jika mengubahnya menjadi assignment edge tidak membentuk cycle** (cycle yang memuat claim edge = unsafe).

Banker's algorithm

n = jumlah thread, m = jumlah resource type. Tiap thread harus menyatakan maksimum di awal, mungkin harus menunggu saat meminta, dan wajib mengembalikan resource dalam waktu terbatas.

Struktur	Ukuran	Arti
Available	m	jumlah instance tiap resource yang bebas
Max	n × m	permintaan maksimum Ti untuk Rj
Allocation	n × m	yang sedang dipegang Ti
Need	n × m	sisa kebutuhan = **Max − Allocation**

Safety algorithm

1. `Work = Available`; `Finish[i] = false` untuk semua i.
2. Cari i dengan `Finish[i] == false` dan `Need_i ≤ Work`. Jika tidak ada, lompat ke langkah 4.
3. `Work = Work + Allocation_i`; `Finish[i] = true`; kembali ke langkah 2.
4. Jika semua `Finish[i] == true` → **safe**.

Resource-request algorithm untuk $Request_i$:

1. Jika `Request_i > Need_i` → **error** (melebihi klaim maksimum).
2. Jika `Request_i > Available` → Ti **menunggu** (resource tidak tersedia).
3. Pura-pura alokasikan: `Available -= Request_i`, `Allocation_i += Request_i`, `Need_i -= Request_i`. Jalankan safety algorithm: safe → kabulkan; unsafe → Ti menunggu dan state lama dipulihkan.

Contoh Banker (A = 10, B = 5, C = 7)

Thread	Allocation (A B C)	Max (A B C)	Need (A B C)
T0	0 1 0	7 5 3	7 4 3
T1	2 0 0	3 2 2	1 2 2
T2	3 0 2	9 0 2	6 0 0
T3	2 1 1	2 2 2	0 1 1
T4	0 0 2	4 3 3	4 3 1

Available = (3, 3, 2). Jejak safety untuk urutan ⟨T1, T3, T4, T2, T0⟩:

```
Work (3,3,2)   T1 need (1,2,2) ≤ Work  →  Work + (2,0,0) = (5,3,2)
Work (5,3,2)   T3 need (0,1,1) ≤ Work  →  Work + (2,1,1) = (7,4,3)
Work (7,4,3)   T4 need (4,3,1) ≤ Work  →  Work + (0,0,2) = (7,4,5)
Work (7,4,5)   T2 need (6,0,0) ≤ Work  →  Work + (3,0,2) = (10,4,7)
Work (10,4,7)  T0 need (7,4,3) ≤ Work  →  Work + (0,1,0) = (10,5,7)   SAFE
```

Urutan aman tidak unik; ⟨T1, T3, T4, T0, T2⟩ juga sah. Work akhir selalu sama dengan total instance (10, 5, 7).

T1 meminta (1, 0, 2): (1,0,2) ≤ Need (1,2,2) dan ≤ Available (3,3,2). State baru: Available = (2, 3, 0), Allocation T1 = (3, 0, 2), Need T1 = (0, 2, 0). Safety menemukan ⟨T1, T3, T4, T0, T2⟩ → **dikabulkan**.

Dari state itu (jawaban pertanyaan di slide):

- **T4 meminta (3, 3, 0)**: 3 > Available A = 2 → **menunggu**, resource belum tersedia.
- **T0 meminta (0, 2, 0)**: lolos cek Need dan Available. Setelah pura-pura dialokasikan, Available = (2, 1, 0) dan tidak ada thread dengan Need ≤ Work (T1 butuh B = 2, T3 butuh C = 1, sisanya lebih besar) → **unsafe, T0 menunggu**.

Deadlock detection

Sistem dibiarkan masuk deadlock, lalu dideteksi dan dipulihkan.

- **Satu instance per resource**: pakai **wait-for graph** (simpul hanya thread; Ti → Tj jika Ti menunggu resource yang dipegang Tj; didapat dengan menghapus simpul resource dari RAG). Ada cycle = ada deadlock. Deteksi cycle butuh $O(n^2)$.
- **Banyak instance**: struktur Available (m), Allocation (n × m), dan **Request** (n × m, permintaan saat ini).

Algoritma deteksi (mirip safety, tapi pakai Request, bukan Need):

1. `Work = Available`. `Finish[i] = false` jika `Allocation_i ≠ 0`, selain itu `true`.
2. Cari i dengan `Finish[i] == false` dan `Request_i ≤ Work`. Jika tak ada, ke langkah 4.
3. `Work = Work + Allocation_i`; `Finish[i] = true`; kembali ke langkah 2.
4. Jika ada `Finish[i] == false` → sistem deadlock, dan **setiap Ti dengan Finish false ikut deadlock**.

Kompleksitas **O(m × n^2)**. Bedanya dengan safety: detection optimistis (menganggap thread tidak akan meminta lagi), safety pesimistis (memakai Need maksimum).

Contoh (A = 7, B = 2, C = 6), Available = (0, 0, 0):

Thread	Allocation	Request
T0	0 1 0	0 0 0
T1	2 0 0	2 0 2
T2	3 0 3	0 0 0
T3	2 1 1	1 0 0
T4	0 0 2	0 0 2

- ⟨T0, T2, T3, T1, T4⟩ membuat semua Finish = true → **tidak deadlock**.
- Jika T2 meminta tambahan satu C (Request T2 = 0 0 1): resource T0 bisa diambil kembali (Work = 0 1 0), tapi tidak cukup untuk siapa pun → **deadlock pada T1, T2, T3, T4**.

Kapan menjalankan deteksi bergantung pada seberapa sering deadlock terjadi dan berapa thread yang harus di-rollback (satu per cycle yang terpisah). Jika dijalankan sembarang waktu, sulit menentukan thread mana yang "menyebabkan" deadlock.

Recovery

- **Process termination**: hentikan semua thread yang deadlock (mahal), atau hentikan satu per satu sampai cycle hilang (overhead deteksi ulang). Urutan pilih korban: prioritas, sudah berapa lama dan berapa lama lagi, resource yang sudah dipakai, resource yang masih dibutuhkan, berapa thread yang harus dihentikan, interaktif atau batch.
- **Resource preemption**: **selecting a victim** (minimalkan biaya), **rollback** (kembali ke safe state dan restart dari sana), dan hindari **starvation** dengan memasukkan jumlah rollback ke faktor biaya.

BAB 18

18 Virtual Machine

INTISARI

Virtualisasi mengabstraksi hardware satu komputer menjadi beberapa lingkungan eksekusi. **Host** = hardware asli; **VMM/hypervisor** = pembuat dan pengelola VM yang memberi antarmuka identik dengan host (kecuali paravirtualization); **guest** = biasanya OS yang mendapat salinan virtual dari host.

Definisi formal (IBM, 1972)

1. VMM menyediakan lingkungan yang pada dasarnya identik dengan mesin asli.
2. Program di dalamnya hanya mengalami sedikit penurunan performa.
3. VMM memegang kendali penuh atas resource sistem.

Akhir 1990-an CPU Intel cukup cepat untuk virtualisasi PC; Xen dan VMware lahir dari sana.

Jenis VMM

Jenis	Ciri	Contoh
Type 0	di firmware; tiap guest dapat hardware khusus ("partition", "domain"); fitur lebih sedikit; bisa virtualisasi bertingkat	IBM LPAR, Oracle LDOM
Type 1	OS khusus yang jalan langsung di hardware, di kernel mode; menyediakan driver, CPU dan memory management; "OS datacenter"	VMware ESX, Citrix XenServer, Joyent SmartOS
Type 1 (OS umum)	OS biasa yang juga berfungsi sebagai VMM; guest diperlakukan seperti proses	Windows Server + Hyper-V, RHEL + KVM
Type 2	aplikasi biasa di atas host OS; host bahkan tak tahu ada VMM; performa lebih rendah, tapi host tak perlu diubah	VMware Workstation/ Fusion, Parallels, VirtualBox
Paravirtualization	guest **dimodifikasi** agar bekerja sama dengan VMM; bukan duplikasi hardware persis; I/O lewat circular buffer di shared memory; perubahan page table lewat *hypercall*	Xen (versi awal)
Programming-environment	VM untuk bahasa, bukan hardware nyata	JVM, .NET CLR
Emulator	menerjemahkan **semua** instruksi dari CPU guest ke CPU host; bisa beda arsitektur; jauh lebih lambat	emulator konsol game, QEMU
Application containment	bukan virtualisasi; satu kernel, aplikasi dipisah dalam zona dengan network stack, user, dan jatah CPU/memori sendiri	Solaris Zones, BSD Jails, AIX WPARs (mirip Docker)

Manfaat

- Host terlindungi dari VM, dan VM terlindungi satu sama lain (virus sulit menyebar); berbagi lewat shared volume atau jaringan.
- **Freeze/suspend**, pindah atau salin lalu resume; **snapshot** dan restore; **clone**.
- Banyak OS berbeda di satu mesin: konsolidasi server, pengembangan dan testing, riset OS.

- **Templating**: satu image OS + aplikasi jadi banyak instance.
- **Live migration**: VM yang sedang berjalan dipindah antar host tanpa memutus user.
- Semua fitur ini digabung = dasar **cloud computing**.

Building blocks

- **VCPU**: struktur yang menyimpan state CPU "menurut guest"; dimuat/disimpan saat guest di-context-switch.
- **Trap-and-emulate**: guest berjalan di *real user mode* dengan dua mode virtual (virtual user dan virtual kernel). Privileged instruction di user mode memicu trap → VMM menganalisis, mengeksekusi atas nama guest, lalu mengembalikan kontrol. Kode user guest berjalan secepat native; kode kernel guest lebih lambat.
- **Binary translation**: untuk CPU tanpa pemisahan privileged/non-privileged yang bersih (x86 lama; mis. `popf` tidak memicu trap di user mode). Jika VCPU di user mode, instruksi jalan native. Jika di kernel mode, VMM membaca beberapa instruksi ke depan: instruksi biasa jalan native, *special instruction* diterjemahkan. Hasil terjemahan di-cache; boot Windows XP ≈ 950.000 terjemahan × 3 µs ≈ 3 detik (≈ 5% lebih lambat).
- **Nested page tables (NPT)**: guest punya page table sendiri, VMM menyimpan NPT per guest dan menjadikannya page table aktif saat guest jalan. Perubahan page table guest dicerminkan VMM. Kelemahannya lebih banyak TLB miss.
- **Hardware assistance**: Intel **VT-x** (2005) dan AMD **AMD-V** (2006) menambah mode *host* dan *guest*, menghilangkan kebutuhan binary translation; kemudian dukungan hardware untuk NPT, DMA, dan interrupt.

Siklus hidup VM

Dibuat VMM → diberi resource (core, memori, jaringan, storage; di Type 0 biasanya dedicated) → dihapus saat tak diperlukan. Karena mudah dibuat, bisa terjadi **VM sprawl** (terlalu banyak VM yang sulit dilacak).

Virtualisasi dan komponen OS

- **CPU scheduling**: sistem single-CPU pun jadi seperti multiprocessor karena tiap guest punya VCPU. Jika CPU fisik cukup, VCPU bisa dedicated; biasanya terjadi **overcommitment** dan VMM memakai algoritma scheduling standar + fairness. Akibat *cycle stealing*, time slice 100 ms di guest bisa molor jadi 1 detik dan jam guest meleset; VMM menyediakan aplikasi integrasi di guest untuk memperbaiki jam.
- **Memory management** (contoh VMware ESX):
  1. **Double paging** — VMM memindahkan sebagian halaman guest ke backing store walau guest mengira halaman itu di frame fisik.
  2. **Balloon driver** — pseudo-device driver di guest yang "menggembung" mengalokasi memori agar guest melepas halaman, atau "mengempis" agar guest punya memori lebih.
  3. **De-duplication** — halaman identik di beberapa guest dipetakan ke satu halaman fisik.
- **I/O**: makin sedikit campur tangan hypervisor makin baik; opsi direct device access, DMA pass-through, direct interrupt delivery (butuh dukungan hardware). Jaringan: **bridging** (guest langsung di jaringan) atau **NAT** (alamat guest disembunyikan VMM).
- **Storage**: Type 1 menyimpan root disk guest sebagai *disk image* di file system milik VMM; Type 2 sebagai file di file system host. Salin file = guest baru; pindah file = pindah guest. Konversi **P-to-V** (disk fisik → format VMM) dan **V-to-P** (sebaliknya).

Live migration

1. VMM sumber membuka koneksi ke VMM tujuan.
2. Tujuan membuat guest baru (VCPU, dll.).

3. Sumber mengirim semua halaman memori **read-only**.
4. Sumber mengirim semua halaman **read-write** dan menandainya *clean*.
5. Langkah 4 diulang untuk halaman yang menjadi *dirty* selama pengiriman.
6. Saat siklus 4–5 sudah sangat singkat, sumber membekukan guest, mengirim state VCPU terakhir dan halaman dirty terakhir, lalu tujuan menjalankan guest. Setelah tujuan konfirmasi, sumber menghentikan guest.

Contoh

- **VMware Workstation**: Type 2 di x86; tiap guest mendapat CPU, memori, disk, dan NIC virtual. Disk guest bisa disk fisik atau file di host.
- **JVM**: programming-environment virtualization (Sun, 1995), *write once, run anywhere*. Kode Java dikompilasi menjadi **bytecode** `.class` yang netral arsitektur, dimuat class loader, dijalankan JVM (yang dikompilasi per arsitektur). Ada garbage collection; **JIT compiler** mengubah bytecode menjadi kode native dan menyimpannya di cache.

Riset virtualisasi

- **Unikernel**: aplikasi + library + layanan kernel yang dipakai dikompilasi menjadi satu binary dengan satu address space; attack surface dan footprint lebih kecil.
- **Partitioning/separation hypervisor** (Quest-V, Xtratum, Siemens Jailhouse): resource fisik dibagi penuh tanpa overcommit; tiap tugas di VM sendiri, cocok untuk real-time dan safety-critical (robotika, mobil otonom, IoT).

BAGIAN II

Praktikum

Referensi cepat untuk lab: bahasa C dari dasar sampai pointer dan fork, Bash scripting, serta grep, sed, dan awk.

P1	Bahasa C — Dasar
P2	Bahasa C — Pointer, Array, fork
P3	Bash Scripting
P4	grep, sed, awk

P1

PRAKTIKUM 1

Bahasa C — Dasar

INTISARI

C adalah bahasa tingkat menengah buatan Dennis Ritchie (Bell Labs, 1972): terkompilasi, dekat dengan mesin (bisa mengatur memori langsung), dan dipakai untuk Linux, database, serta mikrokontroler. Tiap pernyataan diakhiri `;`, program selalu mulai dari `main()`, dan C peka huruf besar-kecil.

Setup dan alur kompilasi

```
sudo apt update && sudo apt install build-essential   # gcc, make, libc
gcc --version
mkdir latihan-c && cd latihan-c && nano hello.c

gcc hello.c -o hello        # tanpa -o, hasilnya a.out
./hello
gcc -Wall latihan.c -o latihan && ./latihan    # -Wall = tampilkan semua warning
```

Empat tahap di balik `gcc`:

1. **Preprocessing** — memproses `#include` dan `#define`.
2. **Compiling** — C → assembly.
3. **Assembling** — assembly → object file `.o`.
4. **Linking** — object file + library → satu executable.

Struktur program

```
#include <stdio.h>      // pustaka standar: printf, scanf

int main() {            // titik awal program
    printf("Halo, C!\n");   // \n = pindah baris
    return 0;           // 0 = selesai tanpa error
}
```

Tipe data dan format specifier

Tipe	Isi	Format
`int`	bilangan bulat	`%d`
`float`	desimal	`%f` (`%.2f` = 2 angka di belakang koma)
`double`	desimal presisi tinggi	`%lf`
`char`	satu karakter, kutip tunggal `'A'`	`%c`
`char[]`	string, kutip ganda `"teks"`	`%s`

Input dengan scanf

```
int panjang, lebar;
printf("Panjang: ");
scanf("%d", &panjang);      // & = alamat variabel tujuan
scanf("%d", &lebar);
printf("Luas = %d\n", panjang * lebar);
```

CATATAN

Lupa `&` di `scanf` adalah kesalahan paling umum (kecuali untuk string `char[]`, karena nama array sudah berupa alamat).

Operator hitung: `+ - * / %` (sisa bagi). Pembagian dua `int` hasilnya `int` (dibulatkan ke bawah).

Percabangan

```
if (nilai >= 75) {
    printf("Lulus\n");
} else if (nilai >= 60) {
    printf("Remedial\n");
} else {
    printf("Belum lulus\n");
}

switch (grade) {
    case 'A': printf("Sangat baik\n"); break;
    case 'B': printf("Baik\n"); break;
    default:  printf("Perlu belajar\n");
}
```

Perbandingan: `==` `!=` `>` `<` `>=` `<=`. Ingat `==` untuk membandingkan dan `=` untuk mengisi. Tanpa `break`, eksekusi `switch` "jatuh" ke case berikutnya.

Perulangan

```
for (int i = 1; i <= 5; i++) {       // nilai awal; syarat; penambah
    printf("Angka ke-%d\n", i);
}

int i = 1;
while (i <= 5) {                     // diulang selama syarat benar
    printf("Angka ke-%d\n", i);
    i++;                             // tanpa ini = infinite loop
}
```

Kunci latihan mandiri

Latihan	Inti solusi
Perkenalan diri	tiga `printf` dengan `\n`
Kalkulator	`scanf("%lf %lf", &a, &b);` lalu cetak `a+b`, `a-b`, `a*b`, `a/b`; cek `b != 0`
Ganjil/genap	`if (n % 2 == 0)` genap, selain itu ganjil
Celsius → Fahrenheit	`f = c * 9.0 / 5 + 32;` (pakai `9.0` agar tidak pembagian integer)
Tabel perkalian	`for (int i = 1; i <= 10; i++) printf("%d x %d = %d\n", n, i, n*i);`
Nilai terbesar dari tiga	`max = a; if (b > max) max = b; if (c > max) max = c;`

Tips: baca error compiler dari yang paling atas (sisanya sering ikut hilang), dan kompilasi sesering mungkin.

P2

PRAKTIKUM 2

Bahasa C — Pointer, Array, fork

INTISARI

Pointer menyimpan **alamat** variabel lain; nama array adalah alamat elemen pertamanya; `fork()` menggandakan proses sehingga kode setelahnya jalan dua kali.

Pointer

Simbol	Arti	Contoh
`&x`	ambil alamat x	`int *p = &x;`
`*p` **di deklarasi**	p adalah pointer ke int	`int *p;`
`*p` **di ekspresi**	*dereference*: nilai di alamat p	`*p = 10;` mengubah x
`NULL`	pointer tidak menunjuk ke mana pun	`int *p = NULL;`

Guna utama: fungsi bisa **mengubah variabel milik pemanggil** (C selalu *pass by value*, jadi yang dikirim adalah alamatnya).

```c
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int x = 5, y = 10;
    swap(&x, &y);
    printf("x=%d y=%d\n", x, y);   // x=10 y=5
}
```

Array dan aritmetika pointer

- `nilai` (nama array) = `&nilai[0]`, bisa langsung dipakai sebagai pointer.
- `p + i` = alamat elemen ke-i; `*(p + i)` = nilainya, **selalu sama** dengan `nilai[i]`.
- Pointer bergerak per ukuran tipe, bukan per byte: `p + 1` pada `int*` maju 4 byte.

```c
int nilai[5] = {10, 20, 30, 40, 50};
int *p = nilai;
for (int i = 0; i < 5; i++)
    printf("nilai[%d]=%d (ptr: %d)\n", i, nilai[i], *(p + i));
```

fork(), exec(), wait()

- `fork()` menduplikasi proses yang sedang berjalan menjadi dua proses identik. Kode **setelah** `fork()` dijalankan dua kali: oleh parent dan oleh child.
- Return value: **0** di child, **PID child** di parent, **−1** jika gagal.

- `getpid()` = PID sendiri, `getppid()` = PID parent.
- `exec*()` mengganti isi proses dengan program lain (`execlp("ls", "ls", "-l", NULL);`); jika berhasil, baris setelahnya tidak pernah dijalankan.
- `wait(NULL)` membuat parent menunggu salah satu child selesai; `waitpid(pid, &status, 0)` untuk child tertentu. Tanpa `wait`, child yang selesai menjadi zombie.

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();
    if (pid == 0) {
        printf("Child: PID %d\n", getpid());
    } else if (pid > 0) {
        wait(NULL);
        printf("Parent: PID %d, child PID %d\n", getpid(), pid);
    } else {
        perror("fork");
    }
    return 0;
}
```

CATATAN

Urutan output parent vs child **tidak dijamin** tanpa `wait()`, karena tergantung scheduler.

Menghitung jumlah proses

- n kali `fork()` berturut-turut (tanpa `if`) → **2^n proses** total, termasuk proses awal; baris setelahnya tercetak 2^n kali.
- `fork(); fork(); printf("hi\n");` → "hi" tercetak **4** kali.
- `for (int i = 0; i < 3; i++) fork();` → **8** proses.
- Child mewarisi salinan variabel parent; perubahan di satu proses **tidak** terlihat di proses lain (address space terpisah, *copy-on-write*).

P3 PRAKTIKUM 3

Bash Scripting

INTISARI

Bash (*Bourne Again SHell*) adalah shell default Ubuntu: penerjemah perintah yang bisa dipakai interaktif atau lewat script `.sh`. Script = kumpulan perintah terminal dalam satu file, berguna untuk otomatisasi yang konsisten (backup, update, bersih-bersih file).

Cek shell dan perintah dasar

```
echo $SHELL        # /bin/bash
bash --version     # GNU bash, version 5.x
which bash         # /usr/bin/bash
```

Perintah dasar yang bisa dipakai di script: `pwd`, `ls -l`, `cd`, `cat`, `mkdir`, `rm`.

Struktur dan cara menjalankan

```
#!/bin/bash
# Baris pertama = shebang: jalankan file ini dengan Bash
# Komentar diawali #
echo "Selamat datang di Bash"
date
echo "Direktori: $PWD"
```

1. Buat file: `nano hello.sh`
2. Beri izin eksekusi: `chmod +x hello.sh`
3. Jalankan: `./hello.sh` (atau `bash hello.sh` tanpa chmod)

Variabel dan input

```
nama="Andi"                # TANPA spasi di sekitar =
umur=20
echo "Halo $nama, umur $umur tahun"

read -p "Masukkan kota: " kota
echo "Kamu tinggal di $kota"

echo "User aktif : $USER"   # variabel bawaan: $USER, $HOME, $PWD
TANGGAL=$(date +%Y-%m-%d)   # command substitution: simpan output perintah
```

- Panggil isi variabel dengan `$nama`; bungkus dengan kutip ganda (`"$file"`) agar aman dari spasi.
- Kutip ganda mengekspansi variabel; kutip tunggal `'$nama'` mencetak apa adanya.
- Konvensi: huruf kecil untuk variabel biasa, HURUF KAPITAL untuk nilai tetap (`SUMBER`, `TUJUAN`).

Percabangan

```
read -p "Masukkan nilai: " nilai
if [ $nilai -ge 75 ]; then
  echo "Selamat, kamu lulus!"
elif [ $nilai -ge 60 ]; then
  echo "Kamu harus remedial."
else
  echo "Maaf, belum lulus."
fi

if [ -f "catatan.txt" ]; then echo "File ditemukan"; fi
```

CATATAN

Wajib ada spasi di dalam kurung siku: `[ $a -eq $b ]`.

Operator	Arti	Operator	Arti
`-eq`	sama dengan	`-ne`	tidak sama
`-gt`	lebih besar	`-lt`	lebih kecil
`-ge`	lebih besar atau sama	`-le`	lebih kecil atau sama
`-f`	file ada	`-d`	folder ada
`=` / `!=`	string sama / beda	`-z`	string kosong

Operator angka (`-eq` dst.) hanya untuk bilangan; untuk string pakai `=` dan `!=`.

Perulangan

```
for i in 1 2 3 4 5; do          # atau: for i in {1..5}
  echo "Perulangan ke-$i"
done

for file in *.txt; do
  echo "Ditemukan: $file"
done

angka=1
while [ $angka -le 5 ]; do
  echo "Angka: $angka"
  angka=$((angka + 1))           # aritmetika: $(( ... ))
done
```

`break` keluar dari loop, `continue` lompat ke putaran berikutnya.

Fungsi dan argumen

```
sapa() {                          # definisikan SEBELUM dipanggil
  echo "Halo, $1! Umur $2 tahun."
}
sapa "Budi" 20

echo "Nama script  : $0"
echo "Argumen ke-1 : $1"
echo "Jumlah       : $#"
# dipanggil: ./fungsi.sh Ani 21
```

Variabel	Isi
`$0`	nama file script
`$1`, `$2`, ...	argumen ke-1, ke-2, ... (di dalam fungsi: argumen fungsi)
`$#`	jumlah argumen
`$@`	semua argumen
`$?`	exit status perintah terakhir (0 = sukses)

Contoh praktis: backup otomatis

```
#!/bin/bash
SUMBER="$HOME/Dokumen"
TUJUAN="$HOME/backup"
TANGGAL=$(date +%Y-%m-%d)
NAMA="backup-$TANGGAL.tar.gz"

mkdir -p "$TUJUAN"                       # buat folder jika belum ada

if [ -d "$SUMBER" ]; then
  tar -czf "$TUJUAN/$NAMA" "$SUMBER"     # c=create, z=gzip, f=nama file
  echo "Backup berhasil: $TUJUAN/$NAMA"
else
  echo "Folder sumber tidak ditemukan!"
  exit 1                                 # keluar dengan status error
fi
```

Jadwalkan otomatis dengan `crontab -e`, mis. `0 2 * * * /home/user/backup.sh` (setiap hari pukul 02.00; urutan field: menit jam tanggal bulan hari-minggu).

Tips

- Selalu mulai dengan shebang, beri komentar, dan pakai indentasi.
- Debug dengan `bash -x script.sh` untuk melihat tiap baris yang dieksekusi.
- Lanjutan: array, `case`, sed, awk, cron.

P4

PRAKTIKUM 4

grep, sed, awk

INTISARI

Tiga alat pengolah teks yang membaca input baris demi baris: **grep** untuk mencari, **sed** untuk mengubah, **awk** untuk mengolah per kolom dan menghitung.

Alat	Paling cocok untuk	Bentuk dasar
grep	mencari baris yang cocok pola	`grep [opsi] pola file`
sed	substitusi, hapus, cetak rentang baris (stream editor)	`sed 'perintah' file`
awk	olah per kolom, filter angka, agregasi, laporan	`awk 'pattern { action }' file`

grep

Opsi	Fungsi
`-i`	abaikan besar/kecil huruf
`-v`	balik: tampilkan baris yang **tidak** cocok
`-c`	hitung jumlah baris cocok (bukan isinya)
`-n`	sertakan nomor baris
`-A n`	tampilkan n baris **sesudah** baris cocok
`-E`	extended regex: aktifkan alternatif (tanda pipa), `{n}`, `+`, `?`

```
grep "ERROR" app.log              # ERROR failed to connect / ERROR timeout occurred
grep -c "ERROR" app.log           # 2
grep -n "ERROR" app.log           # 2:ERROR failed to connect
grep -v "INFO" log.txt
grep -A 2 "ERROR" log.txt
```

Regex dasar: `^` awal baris, `$` akhir baris, `.` satu karakter apa saja, `*` ulang 0 kali atau lebih, `[0-9]` salah satu karakter di dalam kurung.

```
grep "^root" /etc/passwd                # baris yang diawali root
grep -E "^(root|admin)" /etc/passwd     # diawali root ATAU admin
grep -E "[0-9]{3}" access.log           # ada 3 digit berurutan
```

sed

Perintah	Fungsi
`s/cari/ganti/`	ganti kemunculan **pertama** di tiap baris
`s/cari/ganti/g`	ganti **semua** kemunculan (global)
`-n '2,4p'`	cetak hanya baris 2 sampai 4 (`-n` mematikan cetak otomatis)
`/pola/d`	hapus baris yang cocok pola
`-i`	ubah file langsung (*in-place*); tanpa `-i` hasil hanya tampil di layar

```
# file.txt: foo apple foo banana
sed 's/foo/bar/' file.txt     # bar apple foo banana  (hanya foo pertama)
sed 's/foo/bar/g' file.txt    # bar apple bar banana
sed '/^#/d' config.txt        # hapus semua baris komentar
sed -n '2,4p' data.txt
```

awk

Simbol	Arti
`$1`, `$2`, ...	kolom ke-1, ke-2, ...
`$0`	seluruh baris
`NF`	jumlah kolom di baris itu (`$NF` = kolom terakhir)
`NR`	nomor baris
`-F:`	ganti pemisah kolom (default spasi/tab) menjadi `:`
`BEGIN { }` / `END { }`	dijalankan sekali sebelum / sesudah semua baris

```
echo "alice 25 jakarta" | awk '{print $1, $3}'   # alice jakarta
awk -F: '{print $1}' /etc/passwd                 # daftar username
awk '{print NR, $0}' data.txt                    # beri nomor baris
awk '{sum += $2} END {print "Total:", sum}' nilai.txt
awk '$3 > 100 {print $1}' dataset.txt            # filter angka
ps aux | awk '$3 > 5.0 {print $2, $11}'          # PID + perintah, CPU > 5%
```

Variabel awk tidak perlu dideklarasikan (`sum` otomatis mulai dari 0), dan operator matematika tersedia langsung, jadi agregasi cepat tidak perlu program C.

Menggabungkan dengan pipe

```
grep "ERROR" app.log | awk '{print $2}' | sort | uniq -c   # hitung jenis error
ps aux | grep python | awk '{print $2}'                     # PID proses python
cat data.txt | sed 's/,/ /g' | awk '{print $1}'             # CSV → kolom pertama
```